

Auditing the ARC-AGI-3 Public Benchmark: Human Baselines, Scoring Rule, and Environments

Ibrahim Mian Shayaan Siddique
Millennium Research
`{ibby,shayaan}@millenniumresearch.ai`

Abstract

ARC-AGI-3 scores an agent by its action count against per-level human baselines, so its results are only as sound as those baselines, the conditions under which they were measured, and the code that compares agents with them. We audit all three for the 25 public environments: we read the released human replays against the shipped code, and check the scoring rule, the benchmarking harness and two environment levels with machine-verified models run differentially against the implementation.

The 340 released recordings reproduce 165 of the 183 published baselines exactly as the upper median of per-play action counts with resets charged, and on the 72 cells that tell the charged and uncharged readings apart the published value is the charged one on 65: humans paid for their resets as agents do. Sixteen cells, in three environments, cannot be reproduced from the release. All 340 recordings reproduce state and level at every one of their 180,496 steps through the shipped environments, 19 of them only under an engine switch the default does not set: on the engine the humans played, a reset never leaves the level, while on the shipped default a reset at the start of a level, or straight after another, returns the player to level 1. Every published baseline is at or above its level’s optimum.

In the harness, the reset it forces after a game over consumes the agent’s per-level budget, so the effective budget is $\lceil 5b \rceil$ minus the number of deaths, and a paired counterfactual shows this can end a level on a real public environment. Its reach is small: applied to the 1,614 level completions in the human replays, the budget cuts off 29, none of them because of those resets. We also report a discrepancy between the documented and the implemented aggregation rule, and an equation in the technical report that permits a per-level score of 132.25% where every other source caps it at 115%. Two environment models agree with the shipped implementation on all 190,560 transitions of their levels. The code, models, preregistrations and derived data are public at pinned commits.

1 Introduction

ARC-AGI-3 [1], a successor to the Abstraction and Reasoning Corpus [6], is an interactive-reasoning benchmark: an agent is placed in a turn-based environment whose rules are stated nowhere, and must discover them. Scores are efficiency ratios. For each level of each environment, the agent’s action count is compared against a human baseline, the ratio is squared and capped, the level scores are weighted by level index, and the environment scores are averaged. The benchmark is unusual among evaluation suites in how much of it is public:

the toolkit, the harness, the example agents, 25 of the 135 environments, and a technical report describing the scoring rule in detail.

That openness makes an audit possible, and an audit worthwhile. A score under this rule depends on three artefacts being what they claim: the environments, the human baselines that agents are measured against, and the scoring code. This paper checks all three, as far as each admits checking.

Much of what decides a score here is data rather than code. Every level score is a ratio against a number measured from people, and the comparison between agents and humans rests on the two having been measured the same way. The human baselines, the recordings they were computed from, and the conditions under which those recordings were made are therefore as much a part of the benchmark as its environments. Section 8 audits them as data: it reproduces the published baselines from the released recordings, identifies the rule that produced them, and replays every recording through the shipped code to recover the conditions under which the humans played.

Our position is not adversarial. Benchmarks are infrastructure, and the useful service an outside reader can perform is to say precisely which parts have been checked, by what method, and what survived. Where we found nothing wrong we say so at the same length as where we found something.

Contributions.

1. An audit of the published human baselines as data (Section 8). The 340 released human replays reproduce 165 of the 183 baselines exactly under one stated rule, settle that human resets were charged, show every baseline at or above its level’s optimum, and leave 16 cells in three environments that the release cannot have produced.
2. The conditions under which the baselines were collected (Section 8.4): the humans played under reset semantics equivalent to the engine’s `ONLY_RESET_LEVELS` switch, on which a reset never leaves the level, while the shipped default turns a reset at a zeroed action counter into a full reset to level 1. All 340 recordings reproduce state and level through the shipped code under that switch, and 19 only under it, exactly the plays that contain such a reset beyond level 1. A harness probe shows the same 59 actions scoring 10.71 with two levels under the humans’ setting and 3.57 with one under the default.
3. A finding in the benchmarking harness (Section 4.6): it spends the agent’s per-level budget on the resets it issues after a game over, so the effective budget is $[5b]$ minus the number of deaths, which ends a level the same policy otherwise completes on a real public environment at its real budget. The exposure is measured across all 183 public levels and the reach on human play across all 1,614 human level completions.
4. A machine-verified model of the published scoring rule, run differentially against the shipped scorer on preregistered planted traces (Section 4). On every planted trace with an oracle value but one, the scorer agrees with our model of the rule as the report’s prose states it, and the one disagreement is an unreachable edge; five documentation problems are reported: the equation’s cap, the budget’s location, the unstated reset accounting, a stale code comment, and an omitted schema field.

- Machine-verified models of two levels of a public environment, agreeing with the shipped implementation on every transition of their complete reachable state graphs (Section 5), with exhaustive state counts for those levels under a stated identity function, checked against the report’s own stated win probability; and reset and level-accounting probes covering all 25 environments (Sections 6 and 7).

2 Scope, and what we did not do

In scope: the 25 public demonstration environments, the toolkit `arcprize/ARC-AGI` at commit `f12822c`, the harness `arcprize/arc-agi-3-benchmarking` at `1aa78da`, the example agents `arcprize/ARC-AGI-3-Agents` at `4743e7d` (all MIT-licensed), the technical report, and the public documentation.

Out of scope, deliberately and throughout: the 55 semi-private and 55 private environments. We made no attempt to recover, probe, scrape or infer them. All environment execution was local and offline. No component of this work is an agent whose purpose is to score on the benchmark. The random player of Section 7 probed reset semantics and won no game; the blind policy of Section 4.6 measured a cost and completed a level in 16 of its 250 runs without attempting to. Nothing was submitted to any leaderboard.

We audit the *shipped local* artefacts at the pinned commits. The server-side scorer that produces the official leaderboard is not observable to us, and we make no claim about it.

3 Method

Four commitments shape everything below.

Reject-only. No automated component of this work emits a verdict of correctness. Each either reports a disagreement or reports that it found none, and a human reads every disagreement and classifies it before it is called anything. “No disagreement on N fixtures” is a statement about N fixtures.

Preregistration. The bar, the predictions, and the reading of each possible outcome were written down and committed before each measurement. The preregistration was committed (`8e0f70b`) before the first measurement (`08b4f2e`), and each later phase’s plan precedes its runs in the same history; commit order in a history we control is evidence of sequence, not proof, and we state it as such. Deviations are dated in a decisions file and disclosed. One preregistered prediction was wrong and is reported as such in Section 4.

Machine-verified models, run differentially. For each rule under audit we write a model in Dafny [5] whose properties are proved rather than tested, compile it to JavaScript, and compare it against the shipped implementation on traces recorded from that implementation. A model carries no `assume`, `{:axiom}`, `{:verify false}` or `{:extern}`, and a proof obligation that times out is treated as unknown, never as discharged. In the models of Section 5 the level constants are extracted from the shipped level data by a script, so a hand edit to them fails the test suite; the transition rules are hand-written and are checked only by the differential comparison.

Claims discipline. Every number in this paper is produced by a script into an artefact and registered in a claims file, and a checker verifies that each value appears both in the text and on a matching line of the log that produced it. The repository ships the scripts, the artefacts and the checker.

4 The scoring rule

4.1 What the rule says

Report v2 §4.1 defines the level efficiency score for level l of environment e , with $a_{l,e}$ the agent’s action count and $h_{l,e}$ the human baseline, and gives the environment score as a level-index-weighted average subject to a cap at the weighted fraction of levels completed. The human baseline is the upper-median (the report’s term) action count of first-time human players; report v1 said second-best, and the change is documented in an April 2026 post [2].

4.2 An inconsistency between the equation and the prose

The report typesets the level score as

$$S_{l,e} = \min\left(1.15, \frac{h_{l,e}}{a_{l,e}}\right)^2,$$

which caps the *ratio* before squaring and therefore admits a per-level score of $1.15^2 = 1.3225$. The same section’s prose says the per-level efficiency is capped “at 1.15x human baseline” and that a level score lies between 0% and 115%; the documentation says the same; the toolkit’s changelog records the change from a 100% cap to a 115% cap; and the shipped code applies `min(score, 115.0)` after squaring. Every artefact except the equation caps the score.

The difference is observable. Every planted trace has five levels with a baseline of 10 on each, so the level weights sum to 15, and a level not listed is left incomplete. Our planted trace P2c has one level solved in 5 actions against a baseline of 10 and three solved in 12: the shipped scorer returns an environment score of 49.333333, the prose reading returns the same, and the equation reading returns 50.483333. A level solved in 9 actions against a baseline of 10 scores 115.0 in the shipped code and 123.46 under the equation.

4.3 The action budget is not part of the scoring rule

Report v2 §4.3.1 states an action budget of five times the human baseline per level, after which the agent is terminated. We find that budget implemented only in the benchmarking harness, where every shipped model configuration sets a multiplier of 5.0 and the per-level budget is enforced in the agent’s termination test. The scorer applies no cutoff at all. Our planted trace P3b, level 1 completed in 51 actions against a baseline of 10 and the other four levels solved at baseline, is scored 93.589645 by the shipped scorer, where treating the budget as part of the rule gives 0.000000.

Neither the documentation’s methodology page nor its competition-mode page mentions the budget. The consequence is narrow but real: a scorecard computed by the toolkit from a run that did not use the official harness — an example agent, a human session, a third-party agent — is scored without the cutoff the report describes.

We verified separately that the harness enforces the budget exactly. Across six scripted runs against the toolkit in offline mode, driving the harness’s own control loop with a scripted model, no level ever exceeded its budget, the harness’s action counter and the scorecard’s agreed on every run, and two runs exited on the budget precisely at its boundary.

4.4 The aggregation denominator

The scoring rule of Report v2 §4.1 governs one environment. Above it the toolkit computes an overall score, and there the denominator is the number of environments the scorecard contains — the number actually played — rather than the number in the set. The report defines the total as “the sum of individual environment scores divided by the total number of environments”, and the documentation’s methodology page describes it as the average of all game scores. Neither says whether “all” means the environments in the set or the environments played, so what follows is a discrepancy between two readings of an underspecified rule, of which the implementation takes one; we report which reading the code takes and what the other would give.

On a planted scorecard covering three environments of a set of 135, each completed at exactly the human baseline on every level, the toolkit reports a total of 100.0 where the documented rule gives 2.22222222, a factor of 45.0. Played over a whole set with every environment producing a card, the two agree exactly, so the difference is precisely the partial-coverage case.

Which runs this reaches is measured rather than assumed. Calling `Arcade.get_scorecard` under each operation mode, with a transport that refuses to open a socket, identifies where the number comes from: `offline` and `normal` compute it locally through this arithmetic, while `online` and `competition` take a *remote fetch* (*server supplies the scorecard*). The official benchmarking harness constructs the toolkit with `operation_mode` set to `OperationMode.ONLINE`, so this arithmetic does not run for an official run at all. The server-side scorer is not observable to us and we make no claim about it, about the official leaderboard, or about any published result.

The exposure is the local path, and it is not hypothetical. The toolkit’s own minimal example constructs `Arcade()` — the default mode is `normal` — plays a single environment and prints `scorecard.score`. Following the quickstart on one environment prints that environment’s score where the report’s definition gives that score divided by the size of the set.

A second aspect sharpens it. Our first reading supposed that covering the whole set is safe; on the local path it is not, because the divisor counts environments that produced a card. Three environments played perfectly plus a fourth that produced no card — a failure, a skip, an id that did not match — gives a total of 100.0 where the documented rule over four gives 75.0. An environment that produces a card and completes nothing, and one opened but never played, both remain in the divisor and give 75.0 correctly. The boundary is between a card that exists and one that does not.

Relatedly, environment information is matched by full game identifier including version. A scorecard recorded against one version and scored against a listing carrying another scores that environment 0.0, with the message *No Matching EnvironmentInfo found*. That moves a score down rather than up, and a version refreshing between a run and its scoring would silently zero it.

4.5 Resets are charged to the agent, and to the humans

A level reset increments both the reset count and the action count in the scorecard, and the action falls on the level in progress, so it lowers that level’s efficiency. On an otherwise perfect five-level play, one reset during each of levels 2 to 5 takes the environment score from 100 to 83.801652893.

No document we found states whether a reset counts as an action. The methodology page defines an action as a discrete interaction that affects the game state and excludes internal operations, which does not settle it. The question had teeth because human participants were explicitly permitted to reset a level at any time, and the report observes that some “reset levels after reaching a solution in order to improve efficiency”. Had the published baselines been gathered without charging human resets while agents are charged for theirs, every ratio would be biased against the agent by an amount depending on how often each side reset. The replays settle it (Section 8.3): the published baselines are the upper median of the toolkit’s own per-play tallies, which charge every level reset, and on the 72 cells (a cell being one level of one environment) where a human reset and the two readings differ, the published value equals the charged reading on 65, the uncharged reading on one, and neither on six. Humans paid for their resets as agents do. What remains is a documentation gap: the rule is symmetric, and nothing says so.

4.6 Resets the agent did not choose

Section 4.5 concerns resets an agent chooses. The benchmarking harness also issues them on its own: after a game over it returns **RESET** rather than consulting the model, because a reset is the only way to continue. Those resets are counted by the harness like any other action and, on the local path, charged by the scorer to the level in progress.

Driving the harness’s own control loop against the toolkit’s fixture game (the test environment the toolkit ships for its own unit tests), an agent that loses a level three times and then solves it chooses 16 actions and is counted 19, and the level is scored on all 19. Against that level’s baseline of 4, being scored on 19 rather than on the 16 the agent chose lowers that level’s score by 29% relative to the score on 16 (the ratio $(16/19)^2$). That fall has a counterpart in the baselines: a human who died also had to reset to continue, and Section 8.3 shows those resets were charged too. What has no counterpart is the budget, since the humans played without one.

A fall in a completed level’s score is the mild consequence. The counter these resets increment is also the one the harness stops on. The per-level budget is $\lceil mb \rceil$ for the harness’s multiplier m , which every shipped configuration sets to 5.0; the ceiling is the harness’s own and is inert for the integer baselines the benchmark publishes, so we write $\lceil 5b \rceil$ throughout. **is_done** ends the level once the counter reaches it; **choose_action** returns a forced action before the ordinary increment, but the routine that records the forced action increments the same counter regardless. A run can therefore be cut off on an action the agent did not choose.

We measured that as a paired counterfactual rather than arguing it from the code. A scripted policy that dies once and chooses 8 actions runs twice against the same game with the same budget, differing in one respect: in the second run the forced reset still happens, still reaches the environment and is still counted by the scorecard, but does not increment the budget counter. At a budget of 8 the shipped harness exits on the budget with the level

incomplete and the environment scoring 0.0, while the counterfactual completes the level and scores 1.316872428. The agent’s play is identical in both. The reset it did not choose is the entire difference between a completed level and a failed one.

This is not a knife edge. The width of the disagreement follows from the mechanism as read from the code, and we wrote the prediction down before measuring rather than assuming it. The counterfactual completes the level as soon as the budget covers the agent’s chosen actions, while the shipped harness additionally needs one action per game over, so the set of budgets on which the two disagree should be exactly as wide as the number of game overs. It is, in every case probed: no denial budgets for an agent that never dies, [8] for one death, [12, 13] for two, [16, 17, 18] for three. The general statement is simpler than the demonstration. *An agent’s effective per-level budget is [5b] minus the number of times it died.* Each death quietly costs an action of allowance on top of the progress it loses, and the exposure widens with every one.

That demonstration uses the toolkit’s fixture game and a budget we set, and a fixture is not the benchmark. It reproduces on a real public environment at the budget the harness derives from that level’s published human baseline, which we neither supply nor override. On `tu93` level 1, whose baseline of 19 gives a budget of 95, an agent whose 95 chosen actions comprise one losing line and then a winning one is cut off with the level incomplete and the environment scoring 0.0, while the same policy with the forced reset uncharged to the budget counter completes the level and scores 0.087046682. The boundary appears on the same real level: at 93 and at 94 chosen actions both runs complete, and only at 95 does the reset decide it. The losing line and the winning line are each replayed on the shipped environment before use.

What the mechanism costs a policy that is not built to trip it is a separate question, and we measured it rather than leaving the contrivance unqualified. A seeded blind policy over each environment’s advertised actions, clicks included with coordinates inside the declared contract, was driven through the harness’s own loop twice per seed, once as shipped and once with the forced reset uncharged; because the sequence is fixed in advance the two runs choose the same actions as far as both go. Across 250 paired runs over all 25 environments and ten seeds, the median share of charged actions that the agent did not choose is 0.69% and the maximum is 6.28%. The six environments that could be played without clicks gave a maximum near 1%; the largest observed tax is on a click-only environment, so measuring only those six would have understated the maximum. We report maxima, not a class comparison.

Where neither run completes a level, which is 234 of the 250, the extra actions the uncharged run earns are exactly the forced resets the shipped run was charged: the mechanism as an identity. In the remaining 16 it does not hold, and should not, since those extra actions carry the uncharged run to a different point in the game.

The outcome result is null, and it is as much a part of this finding as the denial. No level’s completion and no run’s exit reason differ between the charged and uncharged runs, in any of the 250. This is stronger than what six environments could support: there the null could be explained by a policy that never completes a level, whereas here 16 runs do complete one and the outcome still never differs; we do not claim those 16 were near their budgets, only that the trivial explanation no longer covers every run. The practical reach of this mechanism therefore depends on an agent good enough to finish close to its budget.

We measured that reach on the play the baselines come from. Applying the budget rule

to every level a human completed in the released replays, with the charged count against [5b] and the boundary as measured on `tu93` level 1, the budget cuts off 29 of the 1,614 completions, and none of them only because resets after a game over are counted: in every one of the 29 the human’s own chosen actions already exceed the budget. On human-like play the budget binds rarely, and the counted resets never decide it. What the mechanism can still do is end a run, since the harness stops the whole environment when a level’s budget is exhausted and forfeits the levels after it; how often a capable agent finishes that close to its budget is not measured here.

The sweep also cross-checks the exposure measurement across the whole set, and the two agree in both directions: an environment where no loss inside the budget was found never incurs a forced reset in any seed, and one where a loss was found always does in some seed. Both instruments are random play, so this is a consistency check rather than independent confirmation; the exhaustive result on `1s20` level 1 below is the independent one. The comparison has to be made per environment across its seeds rather than per run, because a single seed may complete the level outright and never die; an earlier version of our own check compared one seed and reported a disagreement that was not there.

The mechanism needs a game over, so we asked of all 183 levels of the 25 public environments whether a level can be lost inside its own budget. Each shipped environment is played from that level’s start under the actions it advertises, with click coordinates inside the declared contract, and the action sequence at the first game over bounds the shortest losing line from above; the search is capped at the budget, since a loss that takes longer is not exposure. Every recorded line is replayed, coordinates included, to confirm it ends the game.

On 148 of the 183 levels, that bound falls below the budget, which proves the level can be lost within it, and 23 of the 25 environments have at least one such level. On the exposed levels the median recorded losing line is about a third of the budget and the median number of deaths it affords is three; both are statistics of upper bounds on the shortest loss. “Deaths affordable” is the budget divided by the recorded line’s length: how many copies of that line fit in the budget, not a demonstration that the line repeats after each forced reset. On the remaining 35 no rollout died, and we report those as not established rather than as immunity, since random play failing to lose says nothing about the shortest line.

The first version of this measurement sampled a fixed set of four actions for every environment, including six that advertise only a click, and reported 17 of 25 first levels. Three of its recorded lines used actions the environment does not advertise, two of them entirely; those actions are accepted and do advance the state, but a line an agent is never told it may play is not a played loss. Re-measuring under the advertised actions moves four environments and gives 19 of 25 at level 1, which is the figure we report. We record the correction because the earlier number would otherwise have been published, and because it confirms that an action outside `available_actions` is accepted and counted: it also advances the state and can end a level.

Where we can compare the two instruments they agree. On `1s20` level 1 an exhaustive search puts the true shortest losing line at 129 against a budget of 110, so that level cannot be lost inside its budget at all, and the capped random search correctly finds nothing there. That is why our first attempt to demonstrate the denial there failed, and we record it rather than dropping it.

Applied to the real public set, one such reset costs a level between 0.035% and 3.33% of its budget, and lowers an otherwise perfect completion (every level solved in exactly its baseline, a level score of 100) by a median, in level-score points, of 3.251814028 points across the 183 levels of the 25 public environments. The worst case is the smallest baseline in the set, `sc25` level 1 at $b = 6$, where one forced reset takes a perfect completion from 100.0 to 73.469387755.

Two things this is not. It is not a claim about how the server scores such a reset, which is not observable to us; the budget itself is enforced by the harness, the one its own README calls the official benchmarking agent, online or offline. And it is not a design criticism: the harness has to send something after a game over. What is absent is any statement to an entrant that these resets happen, that they consume the per-level action budget, and that they can end a level the agent would otherwise have completed. The humans’ resets after a game over were forced by the game in the same way and charged in the same way (Section 4.5); the difference lies in the budget alone.

Several negatives from the same instruments are worth recording, because each rules out a way a client can distort its own score. A retried model call never reaches the environment: the harness’s retry path contains no call that does, and across every scripted run the number of action-bearing environment calls equals the number of actions counted; the construction reset below is not an action. One intended action is sent exactly once: driven with a transport that records every attempt and opens no socket, the remote wrapper makes a single attempt under a connection error, a read timeout after the request was sent, a server error and an empty response, so it never has the server count two actions for one intent. A level that advances on the last action the budget permits is not cut off. The frame cap subsamples evenly rather than truncating and always keeps the last frame, so the settled frame is never dropped; whether the humans who set the baselines saw more is not establishable from the published material and we do not compare. And the reset the wrapper issues when the environment is constructed goes on the wire but is counted by neither the harness nor the local scorer, which treats it as beginning a play; the server’s treatment of it is unknown.

One further limit deserves mention without being a finding. Besides the action budget, a run also stops on a wall-clock limit that the report does not describe. It is uniform: none of the twelve shipped model configurations overrides it or the frame cap, and all set the same action multiplier, so the base of twelve hours for a single environment applies to every entrant. We entered the branch by setting the limit to zero rather than by waiting, and confirmed it ends a run, but at twelve hours per environment we could not show it binds.

4.7 An unreachable edge in the scorer

The engine permits a game to advance the level counter twice within a single action. Where that happens, the scorer pairs the k -th recorded level transition with level k and scores the final level as not completed even though the play ends in a win: our five-level trace P8, every level solved at baseline, returns 66.666667 with the first four levels credited and the fifth not, where the documented rule gives 100.000000.

We then asked whether any public environment can trigger it. Statically, all 25 sources contain exactly one `next_level()` call site and none is inside a loop. Dynamically, we found no such action anywhere: not in either complete state graph of Section 5, not in the 1,245,427 transitions enumerated in Section 6, and not in the 479,040 actions played in Section 7. We

report it as a robustness edge that a guard would close, not as a defect with consequences.

4.8 What the scorer gets right

On 14 preregistered planted traces — 13 of which have an oracle value, the fourteenth exercising a case the rule leaves undefined — the shipped scorer agrees with our verified model of the prose rule on 12, both on environment scores and on every per-level score, disagreeing on one: the double-advance edge above. The agreements include the 115 cap, one-indexed level weights, the environment cap at the weighted completed fraction, best-of-plays selection, and a level solved in exactly the baseline. The model itself discharges 50 proof obligations with no errors and no timeouts.

Two smaller documentation items: the code comments beside the cap still say “max 100”, predating the change to 115; and the published response schema for `/api/games` omits `baseline_actions`, a field the endpoint does return and the toolkit reads.

A wrong prediction, disclosed. We preregistered that the harness would issue and count an initial reset, making it one action stricter than the scorer on the first level. It does not: both toolkit wrappers issue that reset inside environment construction, and neither the harness nor the scorecard counts it. We added a further probe to exercise the boundary the original one had missed, and report the corrected picture: the two counts agree.

5 Environment models

5.1 Method

We selected one public environment by a rule fixed in advance: the fewest lines of transition logic among environments with at least six levels and a mechanic described in the report or documentation. Only `1s20` satisfies the last clause — the report shows its first level as a state graph and states a win probability for it — so it was selected even though eleven environments have smaller sources. All 25 environment sources are identifier-obfuscated, so the models are written from behaviour and structure rather than from names.

For each level we extract the constants from the shipped level object by script (walls, start, goal, modifier tiles, energy pickups, step budget, decrement, lives), hand-write the transition rules from the shipped step function at the rule level, and generate a Dafny model. We then enumerate the level’s reachable state graph directly from the shipped game, and compare the compiled model against the implementation on every transition of that graph and on 30 recorded traces.

5.2 Results

Table 1 summarises. States are counted under the enumerator’s raw key (Section 5.4). The three life-copies of that section contain every state except the game-over states, 143 on level 1 and 1,291 on level 2; each level’s single win state, the other absorbing state, lies inside them. Both models verify with no errors and no timeouts, and agree with the shipped implementation everywhere. Because the environments’ rules are stated nowhere, the models are our rule-level reading of the shipped step function, and agreement on every transition

	1s20 level 1	1s20 level 2
States in the complete reachable graph	14,337	34,739
Transitions	56,772	133,788
Minimum actions to complete	13	45
Published human baseline	22	123
Proof obligations discharged (0 errors)	17	26
Disagreements, model vs implementation, all transitions	0	0
Trace steps compared (30 traces)	3,208	1,903
Disagreements on traces	0	0
RESET returns to the level start	500/500	500/500
Actions advancing the level counter by two	0	0

Table 1: Two levels of the public environment 1s20, audited against machine-verified models of their transition rules.

establishes that the implementation has no behaviour outside the modelled rules on the reachable graph, not that those rules are the intended ones.

The verified properties are: the level is winnable from its own start state, witnessed by the shortest action sequence the enumeration found and replayed through the model with every intermediate state asserted equal to the state the shipped game is actually in; no action leaves the legal state space; and reset restores the start. The first is a stronger statement than “the run ends in a win”: if model and implementation parted company anywhere along the winning path, it would not verify.

5.3 The report’s stated win probability

Figure 3 of the report shows the first level of 1s20 as a graph, notes “three repeating states – an artifact of the three-life mechanic”, and states that the win probability for that level is “exactly 1 in 355”. The term is not defined. Our exhaustive enumeration gives a graph that is indeed three copies of one structure, one per remaining life, and under the reading that seems intended — the probability that a uniformly random policy over the four movement actions reaches the win before a game over, starting from the level’s reset state — we compute 0.002813847150161035, that is 1 in 355.38533070027296. That agrees to three significant figures; “exactly” is a rounding. Two other readings we computed, the reciprocal of the state count and the winning fraction of states, both give 1 in 14,337. We report this as a definitional gap rather than an error: the environment agrees with the claim under its natural reading.

5.4 What counts as a state

A state count depends on the identity function used, and the choice is not neutral. At the rule level the three life-copies are the same size up to a single state on both levels: distinct states by remaining life are 4,732/4,731/4,731 on level 1 and 7,400/7,399/7,399 on level 2. Our enumerator’s raw key, which is deliberately finer than the rendered frame, reports 7,400/13,024/13,024 on level 2. The cause is that a life loss re-appends previously consumed pickups to the end of the level’s sprite list, so a run that consumed one and a run that did not

are distinguishable by internal ordering while rendering identically; level 1 has no pickups and is symmetric under both measures. This changes no transition and no absorption probability, but it is a caveat on any published state count for these environments, including the report’s figure and our own: a frame hash, an engine-state hash and a rule-level abstraction give three different answers.

6 Which environments our enumerator can exhaust

Before enumerating anything we measured what is enumerable. Nineteen of the 25 public environments advertise the click action, whose x and y each range over 0 to 63, giving at least 4,096 distinct actions per state, most of them no-ops. Exhaustive enumeration is out of reach for those within any reasonable budget. The remaining six — **ls20**, **tr87**, **tu93**, **g50t**, **re86**, **wa30** — advertise four or five simple actions.

This is a measured property of the advertised action space, taken from the shipped games. It bounds enumeration over that space, which is what we do; a method that quotients actions by their effect, or builds a partial or abstract graph, is not bounded by it, and published agents do build graphs over these environments.

For those six we enumerated level 1 by depth-first search, which holds only the objects along one path and so is bounded by depth rather than by the width of a search layer. Two completed within budget, **ls20** at 14,337 states and **tu93** at 2,609; the other four stopped at a deterministic cap of 150,000 states, which establishes nothing about reachability either way and is reported as such. Across 616,946 states and 1,245,427 transitions: no level was shown unwinnable, **RESET** returned to the level’s start on all 3,000 probes, and no action advanced the level counter by two.

7 Reset behaviour across all 25 environments

Two of the three questions the enumeration answers need no state graph, so they can be asked of the 19 environments enumeration cannot reach: does **RESET** restore the state the level began in, and can one action advance the level counter by two. We answer both by playing each environment at random inside its own advertised action set, sampling click coordinates uniformly within the declared bounds, and probing as we go.

We measure the reset question twice, because two different relations are at stake. *Frame identity* asks whether the rendered grid an agent observes returns to the level’s opening frame. *Engine-state identity* asks whether the game object does. Conflating them turns a property of the instrument into a claim about an environment.

Across 18,663 probes in all 25 environments, the rendered frame after a level reset was identical to the frame the level began with every single time. Engine state matched on 15,440 of the same probes; in seven environments (**cd82**, **ft09**, **m0r0**, **r111**, **sb26**, **sc25**, **sp80**) a small number of the game object’s own attributes survive a level reset. Several are read back by the games’ own code, so they could in principle influence later behaviour, but none is visible in the frame at reset time.

We do not report this as a defect. ARC-AGI-3 states no rules for any environment by design, so an environment that leaves an internal counter alone contradicts nothing. It does bear on a documented assumption: the report’s own graph construction begins “from the

reset state of a selected level”, and published graph-based agents identify states by hashing the frame. Frame identity and engine-state identity are not the same relation, and in seven of 25 public environments they disagree about whether a reset returns to a canonical state. Anyone building on “the reset state of a level” should say which they mean.

The probe issues a reset only while the engine’s per-level action counter is nonzero, so none of its resets is the full reset of Section 8.4; it reached level 2 in seven environments and no further.

Over 479,040 actions in all 25 environments, no action advanced the level counter by two and none moved it backwards outside a full reset.

Why these negatives are readable. A detector that cannot fire proves nothing. Both are tested against deliberately broken games: with state injected that a reset does not restore, the probe reports a mismatch while the frame still matches; with a reset made to leave a visible change, the count of frame matches falls; and with the engine made to advance two levels in one action, the detector catches it.

8 The human baselines

8.1 A check that needs no human data

Every ARC-AGI-3 score is a ratio against `baseline_actions[1]`. A human cannot complete a level in fewer actions than the level’s optimum, so a published baseline must be at least that optimum. Breadth-first search supplies the optimum, and supplies a sound bound on it even when it does not finish: having expanded every state at depth d without reaching a win, it has proved no solution exists in $d + 1$ actions or fewer.

Two things were settled before measuring, either of which would have made a contradiction spurious. The scorer pairs `baseline_actions[level_idx]` with that level’s own action count, computed as a difference between cumulative counts, so the published values are per level. And the published values are not monotone — `ar25` is [32, 50, 75, 37, 89, 159, 233, 73] — so they cannot be cumulative.

8.2 Result

Of 18 levels attempted across the six enumerable environments, six resolved and all six are consistent; none is impossible; twelve were stopped by a time or memory cap after completing depths between 8 and 16 against baselines between 26 and 183, and establish nothing either way.

Table 2 gives the resolved levels. The tightest margin is `tu93` level 1, where the upper-median human used one action more than the optimum on an environment whose rules are stated nowhere.

We then attempted to resolve the remaining levels by changing the search, and could not. Breadth-first holds a whole layer of game objects, so memory was the obvious suspect; we added depth-limited depth-first search with shallowest-depth memoisation, which gives the same completeness guarantee to a stated depth while holding only the objects along one path, and an iterative-deepening form that banks a bound after each completed depth. Memory fell by roughly a factor of seven on `g50t` level 1, where we measured it. It resolved nothing: in

Environment	Level	Minimum actions	Published baseline
ls20	1	13	22
ls20	2	45	123
ls20	3	39	73
tu93	1	18	19
tu93	2	10	16
tu93	3	19	34

Table 2: Exact optima against published human baselines, for the levels where the search completed. Every optimum is replayed through the shipped game and shown to complete its level.

the same time budget breadth-first reached a greater depth on both levels compared, because deepening re-explores, and on levels breadth-first can finish it is strictly better. The binding constraint is not memory but the size of the state space at the depths these baselines occupy, and on those twelve levels no exhaustive search we can tried reaches depth 26 within the caps of Appendix A.6, let alone 183. The human replays settle them from the other side (Section 8.3): a human’s count is an upper bound on the optimum, and every one of the twelve has a human count at or below its baseline.

These optima are proved against the shipped local environment at the pinned version. The replays show that the humans played that code, under one engine switch the default does not set (Section 8.4).

8.3 The replays, and an error of ours

An April 2026 post [2] states that the ARC Prize Foundation open-sourced the Public Demo dataset, “which includes 342 human step-by-step replays” across the 25 public environments. An earlier version of this work, published in its repository, said we could not locate them. That was wrong, and the error was ours. Our scripted check of the Foundation’s public channels found no replay dataset among the **arcprize** GitHub repositories or Hugging Face datasets, and the one link the announcement gives, a link shortener, refused our request with HTTP 403 whether or not it carried a browser user-agent; we reported the refusal as the link not resolving. Two automated blind reviews of that version pointed out that a 403 from a shortener is what a scripted request gets. Opened in a real browser on 5 September 2026, the link resolves to a Google Drive folder holding the dataset: a folder with one subfolder per public environment, an archive of the same, and a ratings file, dated 13 to 15 April 2026. The observation is recorded as such in the repository, with its provenance stated.

We record the error rather than deleting the trace of it, because it is the same class of defect this audit reports in others’ work: an instrument’s limit reported as a fact about the world.

The data. The archive the link resolves to (111,142,305 bytes; its SHA-256 is in the repository) was downloaded by the first author, read from the zip one line at a time, never extracted and never redistributed; nothing from it enters the repository but sorted per-cell counts and tallies. It holds 340 recordings, not 342, one per session, each in the toolkit’s own recording

format: one frame record per action, the first being the construction reset, and a final line carrying the toolkit’s scorecard for the session with per-play cumulative action counts at each level boundary and a reset count. Every frame names the environment version we audited. The parser would split a session at any full reset after its first action; none of the 340 contains one, so recordings, sessions and plays all number 340. The archive carries no participant identifier, so the report’s “first-time players” cannot be filtered for and every statistic below is over all 340. Cells rest on between 2 and 40 plays (median 8), and 21 of the 183 on fewer than 5. Twenty-three recordings, all from March 2026, are from an earlier recorder that wrote action names rather than ids and no construction line; 22 of them complete at least one level.

They are plays of the shipped code. We replayed every recording action by action, with its recorded click coordinates, through the environments in the repository, comparing rendered frame, state and level after every step. All 340 reproduce state and level at every one of 180,496 steps: 321 under the shipped engine as configured, and 19 only under the engine’s `ONLY_RESET_LEVELS` switch (Section 8.4). 284 reproduce the rendered frame at every step under the shipped engine as configured; 15 do so only under the switch; the remaining 41, in four environments, agree on state and level throughout while 6,844 frames differ in decoration — for example, a pixel or two changing colour on most frames of one environment, animation frames in another, a colour-swapped flash frame in a third — which the game logic does not read.

What the published baseline is. For every play and every level it completed we counted actions two ways: *charged*, every action the scorer counts, level resets included and the construction reset excluded; and *uncharged*, level resets excluded. Our charged count agrees with the toolkit’s own tally in the recording’s final line on 316 plays; on the 22 old-recorder plays that complete a level the stored tally is exactly one below ours at every level boundary, and the same recordings replay through the shipped code with no shift, so it is the old tally that is off; two `lp85` cards record a total and a level count but no boundaries. Over the 183 published integers, the upper median of the charged counts reproduces 165 exactly and four more within one; the upper median of the stored tallies reproduces 166 — it gains the two old-tally cells below and loses `lp85` level 6, where the two boundary-less cards drop out and move the median by one; the uncharged reading reproduces 101, with 42 within one. Lower median, plain median and mean do far worse. The documented rule — the upper median of players’ counts — is the rule, and it charges resets. Because it reproduces over all 340 sessions, either every released session was a first play or the published baselines were not restricted to first-time players as the documentation states; the release does not say which. On the 72 cells where a human reset and the two readings differ, the published value equals the charged reading on 65, the uncharged on one, and neither on six; all seven cells that are not the charged reading lie in the three environments the release does not reproduce.

Two level-1 cells carry the old recorder’s tally. The published level-1 baselines for `cn04` and `tr87`, 29 and 54, equal the upper median of the stored tallies and sit one below the upper median of the actions those humans took by the current scorer’s rule, 30 and 55. The discrepancy is one action in two of the 183 cells; we record it because it is exact.

Three environments are not reproduced from the release. Under the rule that fits everywhere else, `g50t` reproduces four of seven cells, `lp85` one of eight and `vc33` one of seven. Eight published values occur in no released play of their cell at all. One is `cn04` level 1, whose value is the old recorder’s tally above. The other seven are two of `g50t`’s — level 4’s baseline of 230 sits over two released plays that took 31 and 52 actions — and five of `vc33`’s seven. No statistic over these recordings produces those seven numbers, so for those environments the release is not the data the baselines were computed from. Splitting the sessions by testing month does not help. One explanation is ruled out: the announcement’s 342 against the archive’s 340 is a shortfall of two sessions, too few to account for seven cells. Another we cannot test: a revision of the environment after its human testing, which a version string that matches would not exclude. We report this as not reproduced, not as wrong.

Every baseline is at or above its optimum. A human count bounds the optimum from above. In all 183 cells the smallest human count is at or below the published baseline, so every baseline is at or above its level’s optimum, the twelve the search could not resolve included. And in every cell our search had bounded from below, no human beat the bound: the two instruments agree.

8.4 The engine the humans played

The engine’s `handle_reset` performs a *full* reset — every level re-cloned, score and level index zero, a new play opened on the scorecard — whenever its per-level action counter is zero and the game is not won, and a level reset otherwise. That counter is zeroed by `set_level`, which runs at the start of every level and inside every level reset. So a `RESET` issued as the first action of a level, or a second `RESET` straight after a first, is a full reset. The engine’s own switch `ONLY_RESET_LEVELS` removes both cases: under it a reset never leaves the level except after a win.

The recordings show which reset semantics the humans played under. Across the 340 plays, 29 resets fall at a zeroed counter, in 24 of the plays: 28 are a reset straight after a reset — a game over, a reset, then another — and one is a reset at the start of a level. On level 1 a full reset and a level reset produce the same frame, state and level, so the replay can tell the two settings apart only beyond level 1, where 24 of those 29 resets fall, in 19 plays. Those 19 recordings are exactly the ones that reproduce the humans’ state and level only under the switch; 15 of them also match every frame under it, and four, in environments whose frames carry decoration, match on state and level. Under the engine the humans played, each of the 29 resets cost one action. Under the shipped default each of the 24 beyond level 1 would have been a trip back to level 1. Whether the humans’ server ran the switch or the guard described below, the trace is the same; what the replays establish is that their resets never left the level.

What that costs under the real harness we measured with the same scripted-agent driver as Section 4.6, on `ls20`: the 13-action level-1 witness, one `RESET` as the first action of level 2, then the 45-action level-2 witness. Under the humans’ setting the reset is a no-op level reset; the play completes two levels and the environment scores 10.714286. Under the default the reset opens a second play at level 1, the level-2 witness is spent on level 1, the card reads one level completed then none, and the environment scores 3.571429; the harness’s own tracker still records one level completed while the frame reports none, so its counter, never reset and

now at 46, is judged against level 1’s budget. An agent that notices and re-clears level 1 recovers the score at the price of a second play and 13 more charged actions.

Which of the two cases an agent can meet through the harness is settled by the harness’s own code. It offers `RESET` to the model on every turn except the first and except immediately after a `RESET`, its own forced one after a game over included, and it executes only actions it offered. So the reset-after-reset case — 28 of the 29 human events — cannot occur through the harness, and the forced reset of Section 4.6, which leaves the counter at zero, cannot be followed by a chosen one. A `RESET` as the first action of a newly reached level is offered, and that is the case the probe above measures.

Whether the server that scores agents runs the switch is not observable to us. The toolkit’s own API server carries a guard for exactly this case — in “competition mode” a reset that would cause a full reset is not sent to the game at all — and the toolkit’s changelog, for its own version 0.9.5 (the toolkit package, distinct from the engine we pin at 0.9.3), fixes scorecards “when `ONLY_RESET_LEVELS` envvar was set”, so the Foundation knows the case and handles it on at least two paths. The benchmarking harness sets neither, and an agent run through it against the shipped environments meets the default. The finding is the asymmetry that can be shown, on the shipped engine: the reset semantics under which every baseline was gathered are not the semantics the shipped default gives an agent run against those environments. The budget finding of Section 4.6 is different in kind: it is a property of the client and holds online as well.

9 Limitations

A reader should hold the following in mind.

1. Everything here concerns the public toolkit, harness and documents at the pinned commits. The server-side scorer behind the official leaderboard is not observable to us and is not claimed to match. This bears hardest on Section 4.4: we report how a public artefact resolves an underspecified rule, not a claim about anyone’s published result, and we do not compute what any particular reported score would have been.
2. “No disagreement” is over the fixtures and transitions we ran, chosen by us. It is not a proof of correctness.
3. Our Dafny models encode our reading of the report and the shipped source. A disagreement could always be the model being wrong, and we treated that as the default explanation throughout.
4. Two levels of one environment are audited with verified models. The state-graph results cover level 1 of six environments, two exhaustively; we could not enumerate the 19 click-based environments. Whether a level can be lost inside its budget is measured for all 183 levels, but that instrument is one-sided: it proves exposure where it finds a loss and proves nothing where it does not.
5. The random play probe won no game and mostly stayed on level 1, so its negatives cover the states it visited. The blind policy of Section 4.6 is likewise random: how

often a capable agent finishes within a few actions of its budget, and so how far the budget finding reaches beyond random play, is not measured here.

6. The replay analysis reads the released recordings as they are. Three environments' baselines are not reproduced from them and are reported as such; the ratings file beside the archive was not used; and what the scoring server does with a reset at a zeroed counter is not observable.
7. A capped enumeration is a sample of a larger graph. Four of the six enumerable environments are represented by the first 150,000 states a depth-first search happened to visit.
8. Several of our own errors were found during the work and are recorded in the repository's decisions file rather than quietly fixed: a state key that ignored non-`ls20` game state, two hashing errors that merged or spuriously split states, an off-by-one in the lower bound, a peak-memory guard that let one heavy environment mark later ones as capped, an exposure measurement that sampled actions six environments never advertise, a cross-check that compared one seed where it needed all of them, a stated HTTP status (429) that no artefact supported, and a shortener's refusal reported as the replays not being locatable. Each invalidated something that was then re-run or cut.
9. What a reader must still trust: Dafny and its JavaScript backend, Node, the vendored toolkit at the pinned commit, and our reading of the English in the report and documentation.

10 Related work

This paper belongs to a line of work that audits a benchmark's reference data rather than competing on it. Northcutt, Athalye and Mueller [7] find label errors in the test sets of ten widely used datasets, frequent enough to make model selection by test accuracy unreliable. Recht et al. [8] rebuild the CIFAR-10 and ImageNet test sets by their original protocols and measure how far accuracy falls on the new ones. Singh et al. [9] show that undisclosed private testing distorts the rankings of a public leaderboard. Raji et al. [10] question what a benchmark can claim to measure, and Reuel et al. [11] assess 25 AI benchmarks against 40 best practices and find large differences in quality. Closest to our subject, Wei et al. [13] review 115 human baselines in foundation-model evaluations and find them rarely rigorous or well documented, and in the Arcade Learning Environment, whose human-normalised scores are the nearest precedent for ARC-AGI-3's, Machado et al. [14] show how much results depend on evaluation protocol and Toromanoff, Wirbel and Moutarde [15] argue that claims of superhuman play depend on which human baseline is used. Documentation standards such as datasheets [12] ask that a dataset record how it was collected; the collection condition recovered in Section 8.4, the reset semantics the humans played under, is the kind of fact such documentation exists to record. We differ from this work in method: every check here is either a machine-verified model run against the shipped implementation or a script whose output is registered against the paper's claims.

Rudakov, Shock and Cowley [3] build hash-identified state graphs over ARC-AGI-3 environments in order to *solve* them, reporting no state counts, no win probabilities and no

verification. Rodionov [4] builds executable Python world models for all 25 public games, checked against recorded observations rather than machine-verified, and does not audit environments for defects. To our knowledge this is the first machine-verified model of an ARC-AGI-3 environment’s rules, the first exhaustive state counts published for specific levels under a stated identity function, and the first check of the published human baselines against proved optima and against the released human replays.

11 Reproducibility and disclosure

The audit repository, released with this paper at <https://github.com/ibrahimmian36/arc-agi-3-audit>, contains the preregistrations, the decisions file recording every deviation and every error of ours, a dated ledger, the generated models, the scripts, the artefacts, a test suite, and a claims file with a checker that verifies each number in this paper against the log that produced it. Environment sources are not redistributed; they are fetched from the ARC API by a script. The human replay archive is not redistributed either; a script downloads it from the storage location the announced link resolves to, recorded in the repository, and the repository carries its size, its hash, and the per-cell counts derived from it, with no recording name, participant identifier or frame. The three audited repositories were archived at Software Heritage at intake.

This work was made public without prior private notice to the ARC Prize Foundation. We record that plainly rather than implying a disclosure process that did not happen. Two considerations decided it. Everything examined here is already public, and every defect is in published material or in the client an entrant runs on their own machine, so nothing in this paper tells a reader something they could not have derived from the same public sources; there is no exploitable secret to withhold. And the findings that change a number are properties of the shipped client, which any entrant can verify against their own run today. We have no reason to think the Foundation would disagree with any of them, and the repository contains everything needed to check every one.

Broader Impact Statement

This paper reports defects in the public artefacts of a benchmark that other people’s results are measured against, so the first effect of publishing it is on a named third party’s software and documentation. We have tried to make that effect proportionate: every claim is bounded by what was measured, every number is produced by a script that ships with the paper, the negatives are reported beside the positives, and our own errors are recorded rather than removed. A reader who wants to check any claim can run it.

The intended benefit is to the people who interpret ARC-AGI-3 scores. An entrant who knows that the harness charges its own resets, and that the baselines charge human resets too, reads a score more accurately than one who does not; the benchmark’s authors can correct what they choose to correct. The method is not specific to this benchmark, and the same instruments would apply to any interactive benchmark whose scoring code and reference data are public.

Two risks are worth naming. The first is that a finding of this kind can be read as an attack on the people whose work it examines; we have written it as a statement of what the

shipped code does and does not do, and it is published together with the evidence rather than as an assertion to be taken on trust. The second is that describing how a budget is spent could be read as a way to score higher. It offers no route to a score an agent has not earned: the charged resets follow game overs, so the only way to avoid them is to play better, and nothing here bears on the private environments, which we never touched. A third risk is that a toolkit total over a subset of environments is reported as a benchmark score, which Section 4.4 shows would overstate it. The human replay data is used without redistribution and reported only as aggregates, so no participant is identifiable from this paper. No models were trained and no new data was collected from people.

Use of AI tools

Large language models (LLMs) were used throughout this work: to write and debug the audit’s code, and to run blind reviews of drafts, with models from two different providers. We have used an LLM to generate portions of the text of this paper, and we take full intellectual responsibility for all of it, including every claim, number and reference. No claim rests on a model’s judgement: every number in the paper is produced by a script in the repository and checked against the paper by the claims checker (Section 3), every reference was checked at its source, and every disagreement the instruments reported was read by a person before it was called anything.

Acknowledgements

The ARC Prize Foundation publishes the toolkit, harness, agents, technical report and documentation openly and under permissive licences. That is what makes an audit like this possible from outside, and it is not the norm. We thank them for it, and note that the great majority of what we checked was exactly what it claimed to be.

References

- [1] ARC Prize Foundation. *ARC-AGI-3: A New Challenge for Frontier Agentic Intelligence*. arXiv:2603.24621, v2, April 2026.
- [2] G. Kamradt (ARC Prize Foundation). *Measuring Human Performance on ARC-AGI-3*. <https://arcprize.org/blog/arc-agi-3-human-dataset>, 14 April 2026.
- [3] E. Rudakov, J. Shock, and B. U. Cowley. *Graph-Based Exploration for ARC-AGI-3 Interactive Reasoning Tasks*. arXiv:2512.24156, December 2025.
- [4] S. Rodionov. *Executable World Models for ARC-AGI-3 in the Era of Coding Agents*. arXiv:2605.05138, v2, June 2026.
- [5] K. R. M. Leino. *Dafny: An Automatic Program Verifier for Functional Correctness*. LPAR-16, 2010.
- [6] F. Chollet. *On the Measure of Intelligence*. arXiv:1911.01547, November 2019.

- [7] C. G. Northcutt, A. Athalye, and J. Mueller. *Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks*. NeurIPS Datasets and Benchmarks Track, 2021.
- [8] B. Recht, R. Roelofs, L. Schmidt, and V. Shankar. *Do ImageNet Classifiers Generalize to ImageNet?* ICML, PMLR 97:5389–5400, 2019.
- [9] S. Singh, Y. Nan, A. Wang, D. D’Souza, S. Kapoor, A. Üstün, S. Koyejo, Y. Deng, S. Longpre, N. A. Smith, B. Ermiş, M. Fadaee, and S. Hooker. *The Leaderboard Illusion*. arXiv:2504.20879, April 2025.
- [10] I. D. Raji, E. M. Bender, A. Paullada, E. Denton, and A. Hanna. *AI and the Everything in the Whole Wide World Benchmark*. NeurIPS Datasets and Benchmarks Track, 2021.
- [11] A. Reuel, A. Hardy, C. Smith, M. Lamparth, M. Hardy, and M. J. Kochenderfer. *BetterBench: Assessing AI Benchmarks, Uncovering Issues, and Establishing Best Practices*. NeurIPS Datasets and Benchmarks Track, 2024.
- [12] T. Gebru, J. Morgenstern, B. Vecchione, J. W. Vaughan, H. Wallach, H. Daumé III, and K. Crawford. *Datasheets for Datasets*. Communications of the ACM 64(12), 2021.
- [13] K. Wei, P. Paskov, S. Dev, M. J. Byun, A. Reuel, X. Roberts-Gaal, R. Calcott, E. Coxon, and C. Deshpande. *Position: Human Baselines in Model Evaluations Need Rigor and Transparency (With Recommendations & Reporting Checklist)*. ICML, PMLR 267:82265–82325, 2025.
- [14] M. C. Machado, M. G. Bellemare, E. Talvitie, J. Veness, M. Hausknecht, and M. Bowling. *Revisiting the Arcade Learning Environment: Evaluation Protocols and Open Problems for General Agents*. Journal of Artificial Intelligence Research 61:523–562, 2018.
- [15] M. Toromanoff, E. Wirbel, and F. Moutarde. *Is Deep Reinforcement Learning Really Superhuman on Atari? Leveling the Playing Field*. arXiv:1908.04683, August 2019.

A Protocols, identities and resources

Everything in this appendix is also in the repository; it is repeated here so that each result can be interpreted from the paper alone.

A.1 Environment identity

Every public environment is identified by the game id the ARC API returns, which carries a version hash (1s20-9607627b), and by the SHA-256 of the source file the API served on 4 September 2026, of which Table 3 gives the first 16 hexadecimal digits. The sources are not redistributed; a reader who fetches the same ids can confirm they hold the same files.

Game id	Levels	Source SHA-256 (first 16)
ar25-0c556536	8	b965d2faaa4a4d91
bp35-0a0ad940	9	e9aecb52c629c3e7
cd82-fb555c5d	6	dafeabff60a3a99b
cn04-2fe56bfb	6	29977e145a0374b5
dc22-fdcac232	6	74abe3523bd6bc8a
ft09-0d8bbf25	6	aa5b54f48a29da9f
g50t-5849a774	7	6f21f2065744e484
ka59-38d34dbb	7	3e8341f4c317ceb9
lf52-271a04aa	10	a6b01973a0905a44
lp85-305b61c3	8	7d10baec7dd29d07
ls20-9607627b	7	298c810da2850d55
m0r0-492f87ba	6	fc3954236f712759
r111-495a7899	6	95d58ebaa8c758e9
re86-8af5384d	8	cf3c1520e17f0b70
s5i5-18d95033	8	341f85bac1fb5ca7
sb26-7fbdac44	8	dbb4877853a8d30f
sc25-635fd71a	6	871a507cf340d1e4
sk48-d8078629	8	b6ca05eed7ba81fe
sp80-589a99af	6	2306d62311a3cc02
su15-1944f8ab	9	a5f91f7c963d6ca6
tn36-ef4dde99	7	43c30052cdeb2300
tr87-cd924810	6	3274657a6499af5c
tu93-0768757b	9	80e41888f9f7b1a0
vc33-5430563c	7	8afa9f55054b2a24
wa30-ee6fef47	9	0c8b63caee2d092d

Table 3: The 25 public environments as audited.

A.2 State identity

The enumerator’s raw key is a 128-bit BLAKE2b digest of, in order: every sprite of the current level in list order (name, tags, position, visibility, rotation and pixel array; order matters because collision scans iterate in order and can break early); the engine’s state name, score and level index; and every instance attribute of the game object in sorted name order, serialised by content to depth six, with dictionary keys and set members ordered by their own content digests rather than by memory address. Excluded are the five per-action bookkeeping fields (`_action`, `_action_complete`, `_action_count`, `_full_reset`, `_next_level`) and the two level containers (the other levels, which no action mutates, and the immutable templates from which a reset re-clones a level). The key therefore covers every mutable field of the game object other than that bookkeeping, so equal keys are identical engine states for every action the enumeration takes. The enumeration issues no `RESET`; a reset’s effect also depends on the excluded per-level action counter (Section 8.4). The engine is deterministic given its state: 23 of the 25 sources use no randomness, and the two that do (`lf52`, `tr87`) seed it from the level index. Equal keys thus have equal futures under those actions, and an enumeration over keys is exhaustive with respect to the engine’s own state. The two hashing errors of Limitation 8 were in the canonicalisation — ordering by memory address, and hashing numeric scalars by type rather than value — and were caught by an address-independence test before any

run reported here. Engine-state identity in Section 7 is equality of this key; frame identity is equality of the rendered grid. The rule-level counts of Section 5.4 use a hand-written abstraction of the level (position, orientation, colour, shape, lives, steps, goals and consumed pickups) and are reported for comparison only.

A.3 Exposure protocol (Section 4.6)

For each of the 183 levels the shipped game is started at that level. Actions are drawn uniformly from the environment’s advertised set; a click carries coordinates drawn uniformly from 0 to 63 in each axis. A rollout stops at `GAME_OVER`, at a win or level advance (not a losing line), or after $\lceil 5b \rceil$ actions, since a loss that takes longer is not exposure. Eight rollouts are run per level, each with its own generator: Python’s `random.Random` seeded with the string "`<game>:<level>:0:<i>`" for $i = 0, \dots, 7$. The shortest recorded losing line is kept with its coordinates and replayed on the shipped game to confirm it ends in `GAME_OVER`. The per-level outputs are the line, its length, how each rollout ended, and the budget.

A.4 Blind-policy sweep (Section 4.6)

For each environment and each of ten seeds, a script of $\sum_l \lceil 5b_l \rceil + 8$ actions (8 spare, so the script outlives any run) is drawn in advance, uniformly over the advertised actions with click coordinates as above, from `random.Random` seeded with "`<game>:<seed>`". The harness’s own control loop runs it against the shipped environment offline, with the budgets the harness derives from the published baselines. The paired run uses the identical script; its only difference is that the increment of the per-level budget counter for a forced reset is undone immediately after the harness applies it, with the reset itself still sent and still counted by the scorecard. A script of that length cannot be exhausted, because chosen actions never exceed the sum of the budgets. The per-run outputs are the chosen and forced counts, the scorecard’s action and reset counts, the levels completed, the exit reason and the environment score.

A.5 Data used and released

Used, not redistributed. The 25 public environments, fetched from the ARC API by a script in the repository and identified in Table 3; the toolkit, harness and example agents at the pinned commits, all MIT-licensed; and the ARC Prize Foundation’s human replay archive, downloaded from the location the announcement’s link resolves to (Appendix A.7). None is redistributed. Readers obtain each from its source, and the repository records the identity of the copy we used (version hash, commit or SHA-256) so that a reader can confirm they hold the same one.

Released. The audit repository (Section 11), under the Apache 2.0 licence: the scripts, the generated models, the preregistrations, the decisions file and the ledger, and every artefact from which the paper’s numbers are read, including the per-cell sorted action counts derived from the replays. The derived counts carry no recording name, participant identifier, timestamp or frame.

Licence of the human replays. The archive carries no licence file; the Foundation’s announcement describes the dataset as open-sourced. We read it and report only aggregates derived from it.

Intended uses. The derived counts support checking the published baselines and any statistic over them; the scripts support re-running every measurement in this paper against the same public artefacts or later versions.

Hosting and maintenance. The repository is public on GitHub, and the audited repositories were archived at Software Heritage at intake. A test suite checks every figure in the paper against the artefact it comes from. A number is corrected by re-running the script that produced it, and errors are recorded in the decisions file rather than silently fixed. The authors maintain the repository.

Hardware. Every measurement ran on one laptop (Apple M2 Pro, 16 GB of memory, macOS); the caps and peak memory of each instrument are stated in this appendix.

A.6 Enumeration and search caps

The two complete `1s20` graphs were enumerated breadth-first under caps of 400,000 states, 1,500 seconds and 3,500 MB of resident memory, and completed well inside them. The level-1 sweep of Section 6 ran depth-first with caps of 150,000 states, 1,200 seconds and 2,500 MB; the baseline search of Section 8 breadth-first with 400,000 states, 120 seconds and 2,500 MB per level; the reset probes of Section 7 with 20,000 actions and 60 seconds per environment at seed 0. Every artefact records the peak resident memory of the run that produced it. All runs were made on one laptop.

A.7 Replay protocol (Section 8.3)

Archive `arc_agi_3_public_demo_human_testing.zip`, 111,142,305 bytes, SHA-256 beginning `99a32ffc`, read from the zip one line at a time. Recording = one JSON object per line; a line with `action_input` is a frame after one action; the final line is the session’s scorecard. A play is split at a frame flagged `full_reset`. Level k , counting from zero, is credited when `levels_completed` first exceeds k ; a level never completed contributes no count. Charged count = actions at that level, level resets included, the construction reset (the first record, when it is a `RESET`) excluded; uncharged = the same without level resets. Upper median of n sorted values = the value at index $\lfloor n/2 \rfloor$. A cell is decisive when at least one human reset there and the charged and uncharged upper medians differ. Replay: the environment is constructed as the toolkit constructs it, each recorded action applied with its recorded x , y ; a recording is classed by the weakest engine setting under which every frame, state and level matches, and “decoration only” requires state and level to match at every step under the switch. A trap event is a `RESET` while the engine’s per-level counter is zero and the game is not won, split by whether the previous action was also a `RESET`. Peak resident memory stayed under 300 MB in both the baseline script and the replay script.

A.8 Tools and proof obligations

Dafny 4.11.0 with its JavaScript backend, Node 24.16, Python 3.12.13, **arcengine** 0.9.3, and the three audited repositories at the commits of Section 2. The supplementary archive records the commit it was built from in its **COMMIT** file. The scoring model’s 50 obligations comprise the definitions of the prose and equation readings and lemmas S1–S6: both caps bound the level score (S1), a level solved in exactly its baseline scores 100 (S2), the level score is monotone non-increasing in the action count (S3), the environment score is capped at the weighted completed share (S4), every level at baseline gives 100 (S5), and nothing completed gives 0 (S6), with the arithmetic lemmas they rest on. Each environment model discharges E1 (the level is winnable by the witness, with every intermediate state asserted equal to the shipped game’s), E2 (no action leaves the legal state space) and E3 (reset restores the start).